

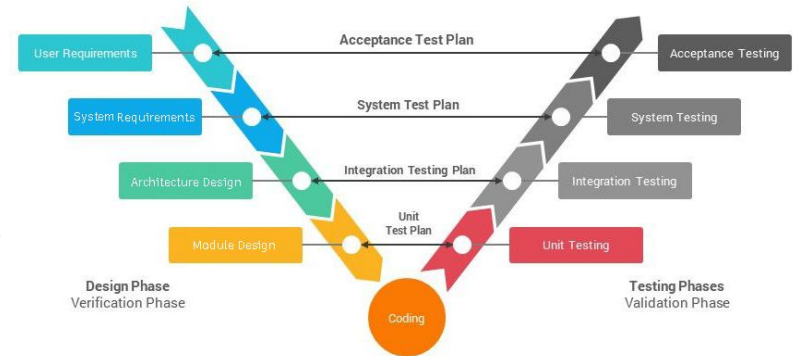


Software Design

Software Engineering

Software Design Engineering

- ❖ *Software design engineering* is the process of defining the architecture, components, interfaces, and other characteristics of a system or component.
- ❖ It involves specifying how the system or component will work, be structured, and meet its requirements.
- ❖ It provides a framework for creating **Software Design Description** documents that describe the design of software systems, helping to ensure that designs meet their requirements and are understandable, maintainable, and testable.
 - An SDD typically includes architectural views, data design, interface design, and detailed design. It helps in understanding the software structure and serves as a blueprint for development.



Software Design Engineering

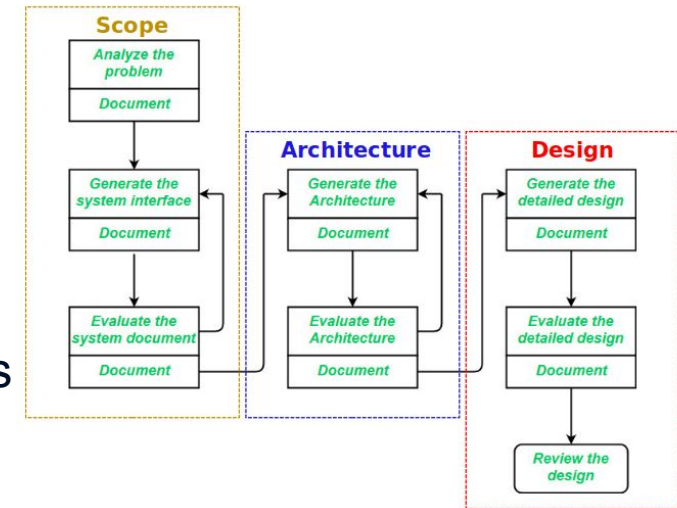
❖ The process is divided into three phases

➤ **Scope** deals with understanding the system, its *boundaries and interfaces* with external systems.

- Top level abstraction models with respect to the inner workings of the system are used. I.e, during this phase, focus is on the interfaces and the internal of the systems are completely ignored and the system is treated as a black box

❖ **Architecture Design:** This involves the high-level structuring of the system, defining its main components and their interactions.

❖ **Detailed(or software) Design:** defines the internal structure and behavior of the system's components, ensuring each part functions as intended



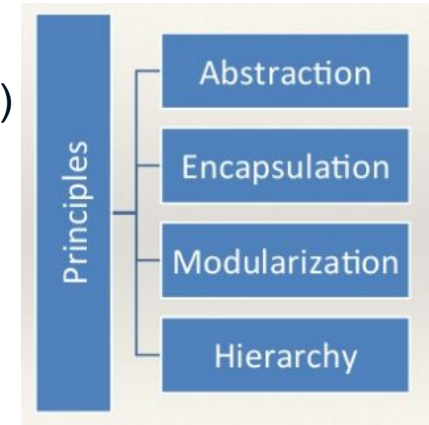
General Design Principles & Considerations

❖ Four fundamental principles:

- PHAME (**P**inciples of **H**ierarchy, **A**bstraction, **M**odularisation, and **E**ncapsulation)

❖ Design considerations:

- **Compatibility** – ability to operate with other products or an older version of itself.
- **Extensibility** – adding new capabilities without major changes to the underlying architecture.
- **Modularity** - well defined, independent components which leads to better maintainability
- **Fault-tolerance** - The software is resistant to and able to recover from component failure.
- **Maintainability** - A measure of how easily bug fixes or functional modifications can be accomplished.
- **Reusability** - ability to use some or all the parts of a software in other projects with little or no modification.
- **Reliability** (Software durability) – ability to perform a required function under stated conditions for a specified period.
- **Robustness** - ability to operate under stress or tolerate unpredictable or invalid input.
- **Security** – Ensuring that applications are protected against threats and vulnerabilities
- **Usability** - must be usable for its target user/audience.
- **Performance** – ability to perform its tasks within a time-frame and memory usage that is reasonable.
- **Portability** - being usable across several different conditions and environments.
- **Scalability** – ability to adapt to increasing data or added features or number of users.



Software Design Engineering

- ❖ **Design Phase:** During this phase, a detailed design of the software is developed based on the requirements. It highlights the need for **traceability** from requirements to design to implementation.
- ❖ **Documentation:** Design decisions, including trade-offs, alternatives considered, and rationale behind the final choices shall be documented
- ❖ **Design Verification:** The software design should be verified to ensure it meets the specified requirements. This includes reviewing design documents and ensuring compliance with design principles.
- ❖ **Design Validation:** Checking that the design will fulfill the intended purpose and meet user needs when implemented.

Software Design Engineering

- ❖ **Iterative Refinement:** An iterative approach to software design is recommended. The design is continually refined based on feedback and evolving requirements.
 - In an Agile context, design is often iterative, incremental, adaptive design and responsive to requirements changing.
 - **Refactoring:** Continuous improvement of the design structure without changing its external behavior, which is a key practice in Agile environments.
- ❖ **Use of Design Patterns:** The use of established design patterns is encouraged to solve common design problems, ensuring that the design is robust and maintainable.
- ❖ **Requirements Traceability:** Maintaining traceability between requirements and design elements. This ensures that all requirements are addressed in the design and can be traced back to specific components.

Software Design Engineering

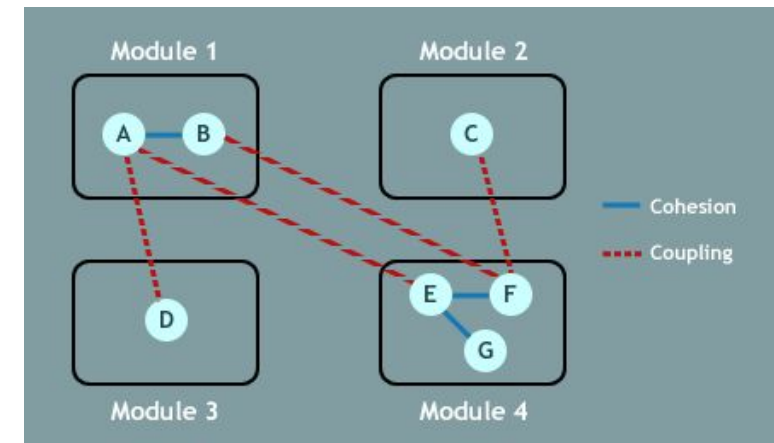
- ❖ **Design-to-Code Traceability:** Maintaining traceability from design to implementation, ensuring that the code accurately reflects the design.
- ❖ **Separation of Concerns:** This principle involves separating the system into distinct sections that each handle a specific aspect of the functionality, such as user interface, business logic, and data management.
 - A common implementation of this principle is a layered architecture (e.g. the OSI model)
- ❖ **Tools and Technologies:** To support the design process and creating design diagrams, maintaining traceability, and ensuring consistency across the design.
- ❖ **Version Control:** For managing changes to design documents and ensuring that multiple designers can collaborate effectively.

Software Architecture Design

- ❖ Is the first stage in the software design process.
- ❖ Is the most abstract design of the system and translates requirements into architecture.
 - Requirements are dynamic, yet some design decision must be taken early in the project!
 - Modular design with high cohesion allows for smoother future modifications.
- ❖ The system is **decomposed** into major **components**.
 - Most decomposition paradigms suggest breaking down a program into parts to **minimize the static dependencies** among those parts, and to **maximize the cohesiveness** of each part.
- ❖ Architecture is the critical link between design and requirements engineering, as it identifies the main structural components in a system and the relationships between them.
- ❖ The overall structure of the system is chosen, but the internal details of components are ignored
- ❖ The output of the architectural design process is an architectural model that describes how the system is organized as a set of communicating components.

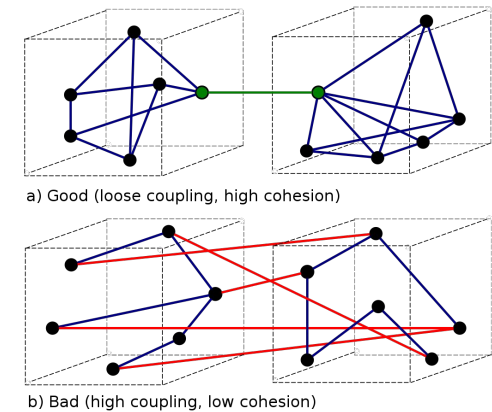
Software Architecture Design

- ❖ A **module** is a self-contained piece of code that handles a well-defined task in a system.
 - A combination of the modules makes up the system.
- ❖ A **component** is made up of modules and provides a function or group of related functions.
- ❖ **Coupling** is the degree of interdependence between software modules.
- ❖ Different types of coupling are:
 - **Content**: modules share code
 - **Common**: modules share global data.
 - **External**: modules share externally imposed data format
 - **Control**: one module controls the flow of others
 - **Stamp**: modules use only parts of their inputs
 - **Data**: modules only share data



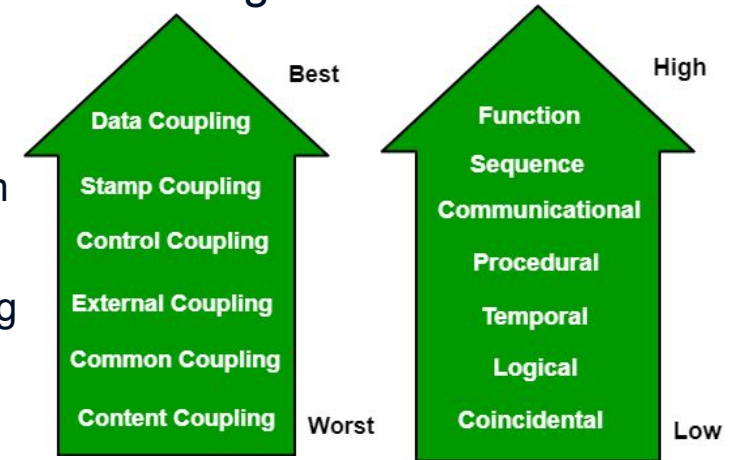
Software Architecture Design

- ❖ **Cohesion** is the degree to which the elements inside a module belong together.
- ❖ Different types of cohesion are:
 - **Coincidental**: when parts of a module are grouped arbitrarily (**worst**)
 - **Logical**: when parts are grouped because they are logically categorized to do the same thing even though they are different by nature. E.g. all mouse and keyboard input handling routines.
 - **Temporal**: parts of a module are grouped by when they are processed. The parts are processed at a particular time in program execution. E.g. a function which is called after catching an exception to close open files and create an error log.
 - **Procedural**: when parts of a module are grouped because they always follow a certain sequence of execution. E.g., a function which checks file permissions and then opens the file.
 - **Communicational**: functionalities working on the same data are grouped together
 - **Sequential**: when the output of one part of a module is the input to the other
 - **Functional**: modules functionalities contribute to a single well-defined task(**best**)

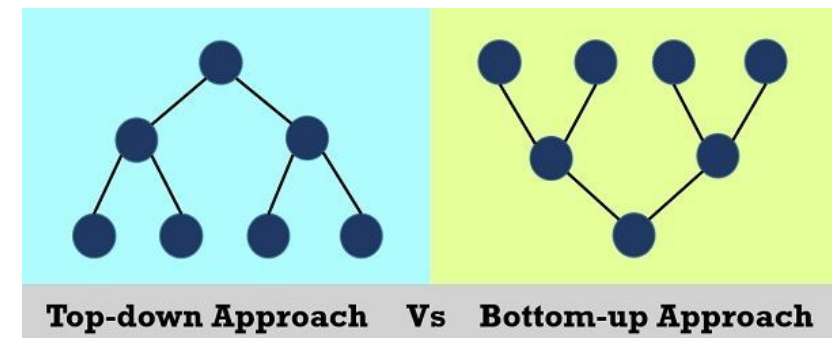


Software Architecture Design

- ❖ Systems with high cohesion and low coupling are considered as well designed.
- ❖ [Top Down vs Bottom Up](#) Design
- ❖ Modern software design approaches usually combine both top-down and bottom-up approaches. Although an understanding of the complete system is usually considered necessary for good design, leading theoretically to a top-down approach, most software projects attempt to make use of existing code to some degree. Pre-existing modules give designs a bottom-up flavor. Some design approaches also use an approach where a partially functional system is designed and coded to completion, and the system is then expanded to fulfill all the requirements for the project.

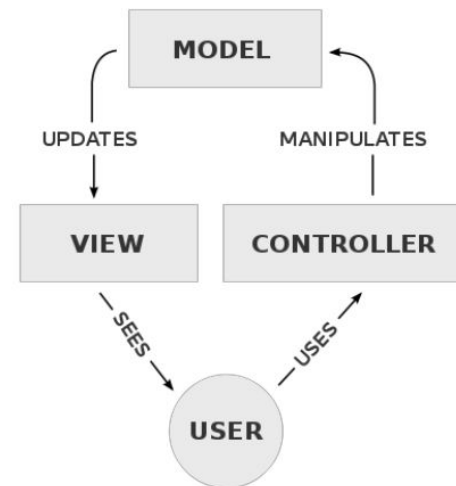


Top-Down	Bottom-Up
Emphasizes on planning and a complete understanding of the system	Emphasizes on early implementation and testing
Stubs to be used for modules yet to be implemented which delays testing ultimate functionality	Integration of all the modules could be concerning, yet code re-use could be increased



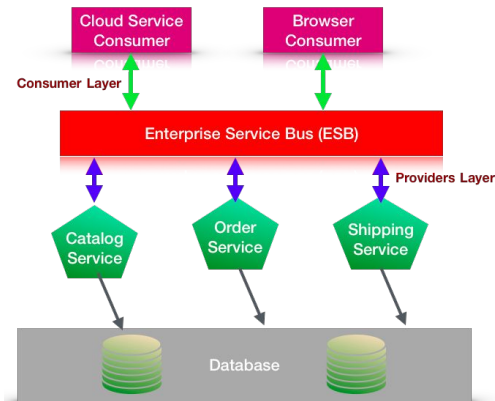
Software Architecture Design

- ❖ Architectural pattern: a stylized and abstract description of a good practice, which has been used and proven for different systems and environments.
- ❖ There are [different architectural patterns](#) and each pattern satisfies certain requirements, the most appropriate one is the one which satisfies most of the requirements.
- ❖ Some examples of Architectural patterns
 - MVC (Model-View-Controller)
 - Layered (e.g. the OSI model)
 - Client-server
 - [Event-driven](#)
 - [Service-oriented](#)
 - Broker
 - And etc.



MVC is mostly used for user interface development. Common in mobile & web app developments. Model or the central component is the app's dynamic data structure. View is the presentation only and Controller accepts input and converts it into commands for model.

Software Architecture Design



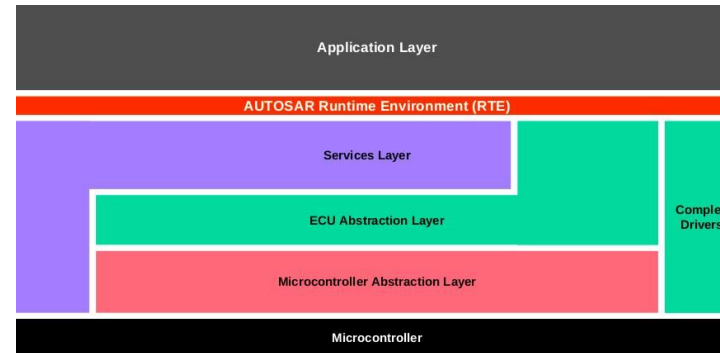
Service-oriented architecture (SOA) pattern

Service-Oriented Architecture:

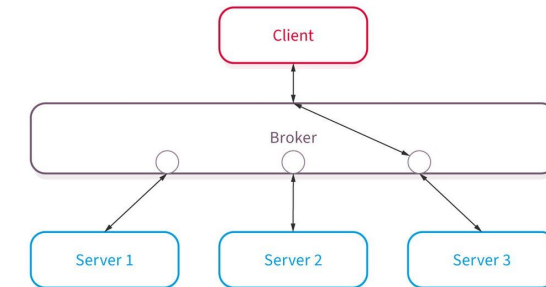
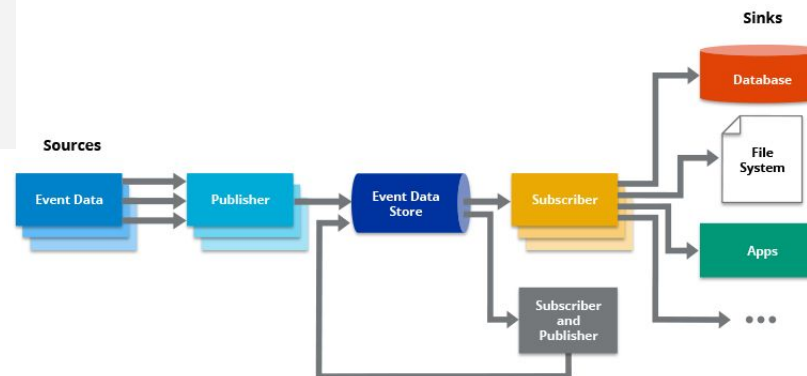
- Clients makes a request
- Clients need to know the service API
- Clients are known to service provider
- Communication is bi-directional

Event-Driven Architecture:

- Event Data Store pushes the event
- Producers and consumer are less coupled
- Producers has no knowledge of consumers
- Communication is unidirectional

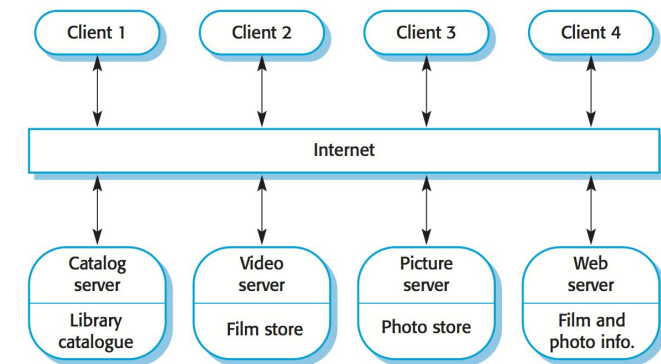


The AUTOSAR architecture is based on a 3-layered architecture model



Broker pattern

Servers publish their data to the broker and the broker sends data to the subscribed clients.



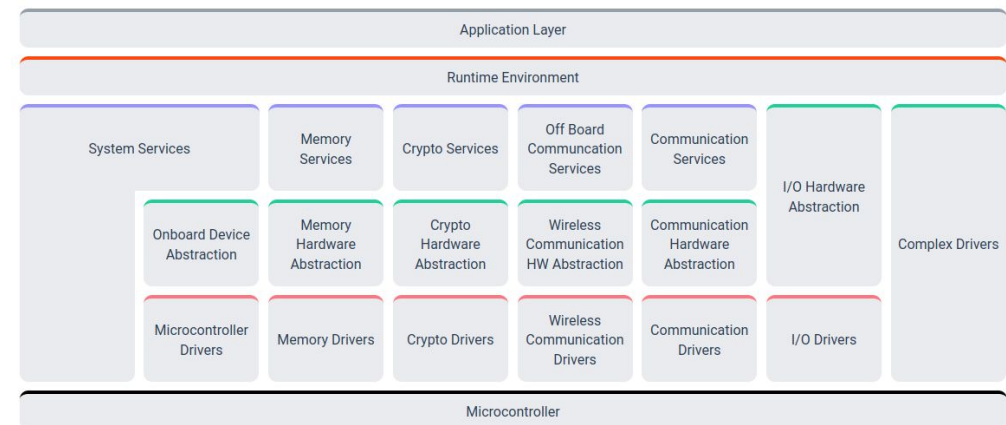
Client-Server architecture

Software Architecture Design - AUTOSAR

❖ AUTOSAR (AUTomotive Open System ARchitecture)

- Industry standard for automotive software architecture
- Developed thru cooperation between most major car and truck OEMs and Tier 1's
- Created to enable reuse, expand supplier base (for BSW) and standardize the industry
- A very complex solution. Only a small part is used in a specific project
- Supports all active technologies
- Support for functional safety
- Classic and Adaptive platforms
- Is used to make portable and reusable software components independent of a specific ECU

ECU: A microcontroller and connected peripherals according to the software/configuration



An example of the classic AUTOSAR

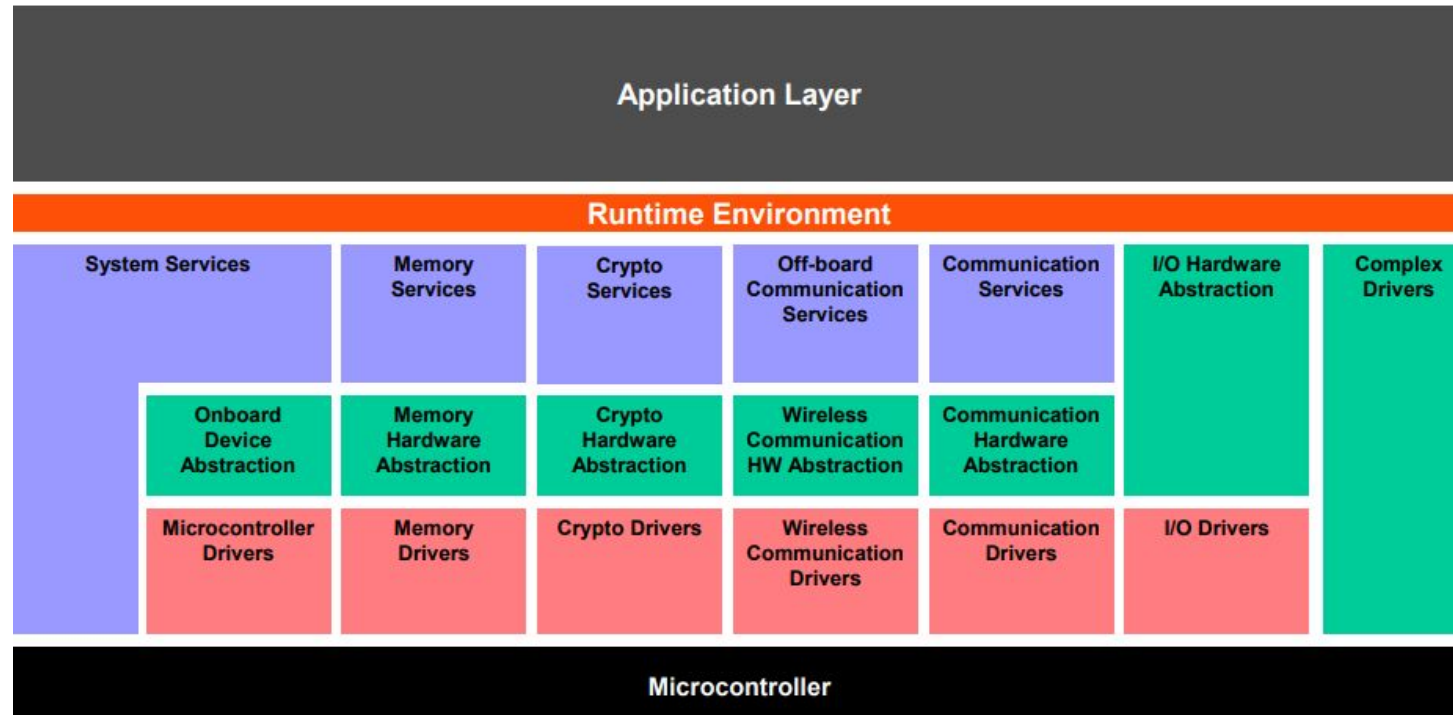
Software Architecture Design - AUTOSAR

- ❖ The AUTOSAR abstracts hardware components from the application layer
- ❖ The AUTOSAR is a layered architecture
- ❖ The AUTOSAR highest abstraction level
 - Application Layer
 - Runtime Environment
 - Basic Software which is divided to:
 - Microcontroller Abstraction Layer
 - ECU Abstraction Layer
 - Services Layer
 - Complex Drivers



Software Architecture Design - AUTOSAR

- ❖ The Basic Software Layer is further divided into functional groups and modules
 - E.g. Drivers, hardware abstractions, memory and communication services and etc.

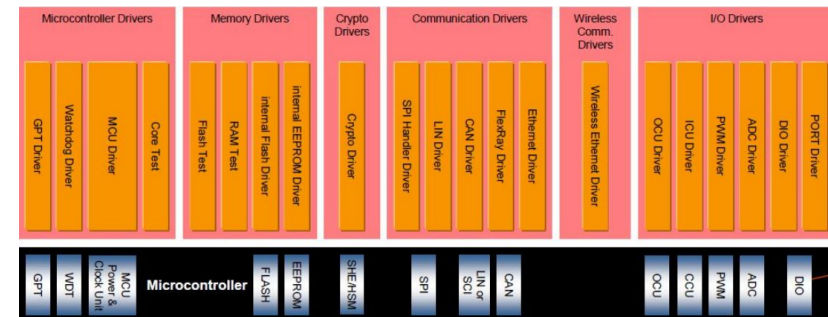
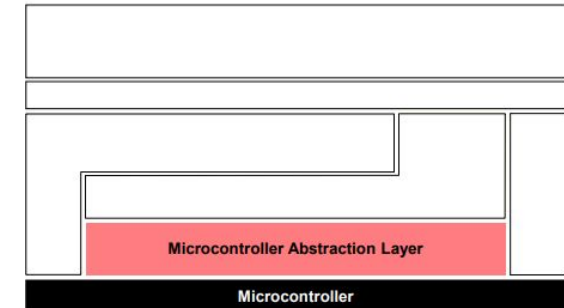


A typical Basic Software architecture and the functional groups that are usually present

Software Architecture Design - AUTOSAR

❖ Microcontroller Abstraction Layer

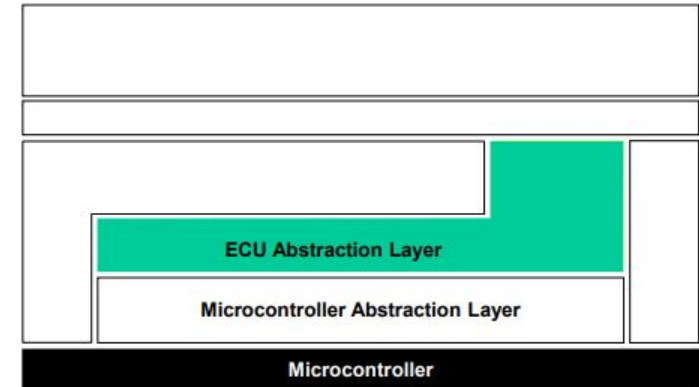
- Different names: BSP / MCAL
- The first layer in the abstraction
- It contains on board device drivers, which are software modules with direct access to the μ C and internal peripherals.
- Makes higher software layers independent of μ C
- The implementation is μ C dependent and the upper interface is standardized and μ C independent
- Examples:
 - I2C, SPI, IO modules and etc. are abstracted in the BSP



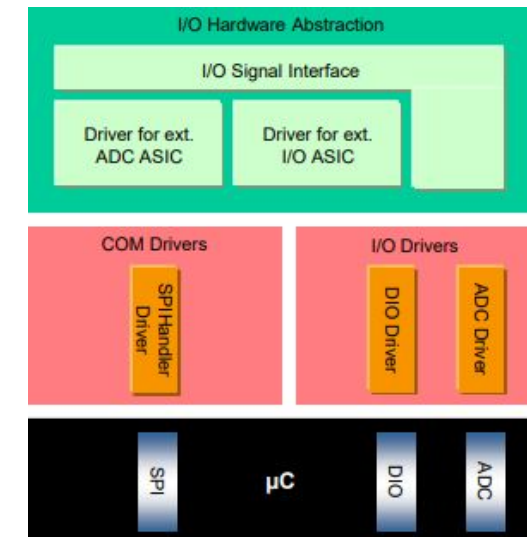
Software Architecture Design - AUTOSAR

❖ ECU Abstraction Layer

- Interfaces the drivers of the MCAL.
- It also contains drivers for external devices.
- It offers an API for access to peripherals and devices regardless of their location (μ C internal/external) and their connection to the μ C (port pins, type of interface)
- Makes higher software layers independent of ECU hardware
- The implementation is μ C independent and ECU dependent
- The upper interface is μ C and ECU hardware independent
- Example: IO Hardware Abstraction
 - Second layer of the HW detachment
 - Abstracts from functional names to actual IO ports
 - E.g. Set_BrkLight_Ind(HIGH) -> BSP_SetDO_2(HIGH)



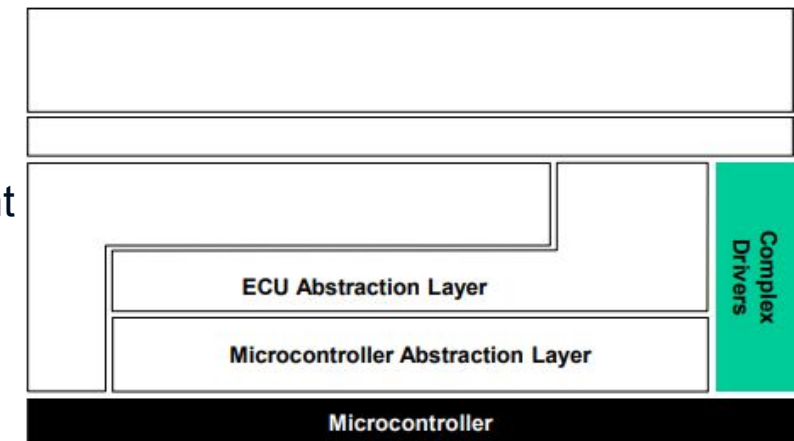
Example



Software Architecture Design - AUTOSAR

❖ Complex Drivers Layer

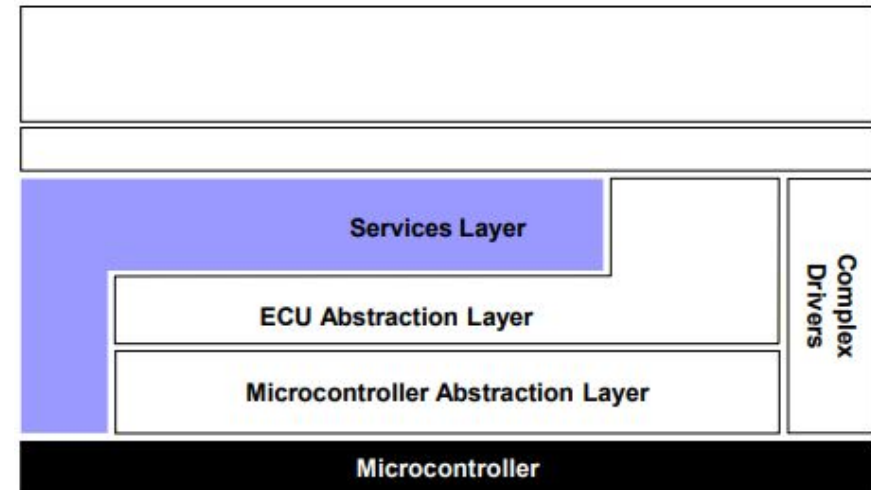
- Provides the possibility to integrate special purpose functionality
- E.g. drivers for devices which:
 - Are non-standard and not specified within AUTOSAR
 - Are hardware dependent and can not be run in application space; E.g. using interrupts
 - Are timing critical and have very high timing constraints and direct access to hardware and optimization are required
- The implementation and the upper Interface
 - Might be application, μ C and ECU hardware dependent
 - But the upper interface fulfills the standard Autosar
- Examples
 - Incremental position detection



Software Architecture Design - AUTOSAR

❖ Services Layer

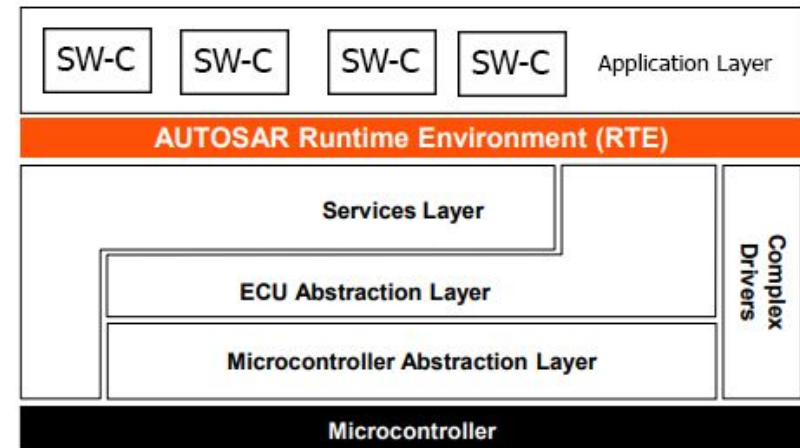
- Is the highest layer of the Basic Software
- Offers all the services needed in the platform
 - Operating system functionality
 - Memory services (NVRAM management)
 - ECU state and mode managements
 - Watchdog management
 - Network communication and management services
 - Diagnostic Services. E.g. Unified Diagnostic Services (UDS)
- The implementation is mostly μ C and ECU hardware independent
- The upper Interface is μ C and ECU hardware independent
- Example: Memory Services



Software Architecture Design - AUTOSAR

❖ Runtime Environment (RTE)

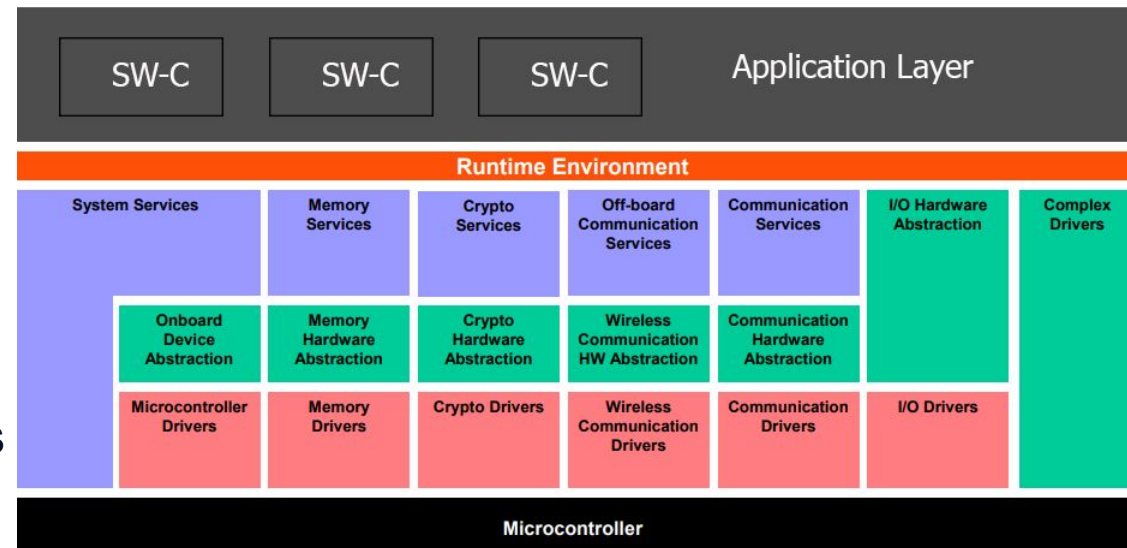
- Makes AUTOSAR Software Components independent from the mapping to a specific ECU.
- Provides stable and fixed interface for the applications
 - Allows reuse of applications between projects
- Provides interaction between software components
- Interfaces defined in ports and interfaces
- Interface towards the Software Component(SW-C)s:
 - Communication, Scheduling, Calibrations, and etc.
- The implementation is ECU and application specific
 - Generated individually for each ECU
- The upper interface is completely ECU independent



Software Architecture Design - AUTOSAR

❖ Application Layer

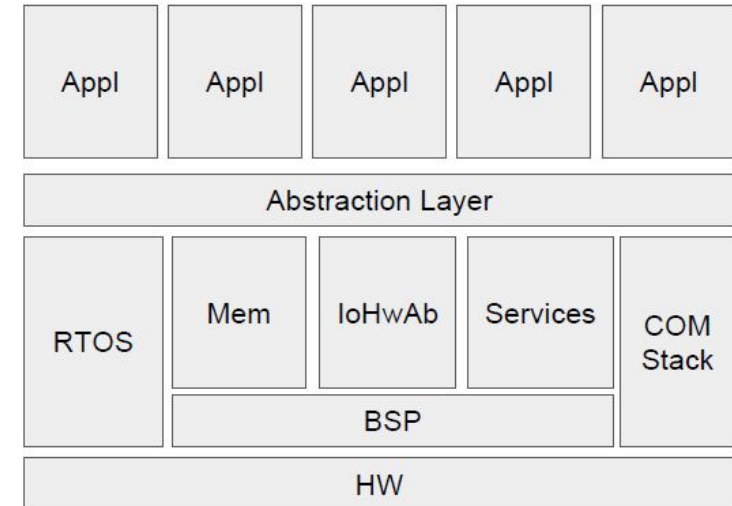
- Completely independent of a specific ECU
- Business logic & value
- Time triggered
- Execution requirements
- Required interfaces & signals
- Calibration and storage requirements
- And etc.
- Example: The Sensor/Actuator AUTOSAR Software Component



Software Architecture Design - Pattern

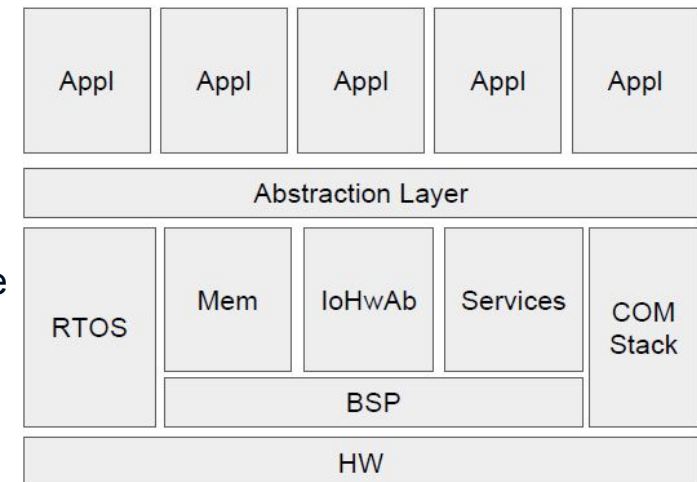
❖ Software Architecture

- BSP/MCAL
 - Abstraction from the hardware
- RTOS
 - Manages the timing of the entire architecture
- Memory Manager
 - Handles Non-volatile memory access, buffering and etc.
 - Loads calibration parameters at startup
- Services
 - Contains generic services like startup and shutdown, mode management and etc.
- IO Hardware Abstraction
 - Abstracts the HW from the abstraction layer. E.g. `set_led_state(uint_8 state) -> digitalWrite(13, state)`
- COM Stack
 - Layered stack to abstract from communication protocols
- Abstraction Layer
 - Provides stable and fixed interfaces for the applications
- Applications
 - Business logic and functional blocks and execution requirements



Software Architecture Design - Pattern

- ❖ Tips for mapping the requirements to the software architecture
 - IO requirements (requirements that specify hardware dependencies)
 - The interfaces are mapped to the IO Hardware Abstraction (IoHwAb)
 - Signal interaction is done via the abstraction layer
 - Network interaction (communication)
 - Network signals are mapped to respective stack
 - Access to signals is achieved via the abstraction layer
 - Application does not care about the source/target network type
 - Only the characteristics matters: timing, value and etc.
 - Application Architecture
 - Use common sense
 - Bundle by functionality
 - Separation of duties
 - E.g. Bundle everything related to ambient temperature into one block



Software Architecture Design - Requirements & Mapping

❖ Example: some system requirements

[REQ-01]: It shall be possible to read the <soil_moisture_min> and <soil_moisture_max> parameters from CAN

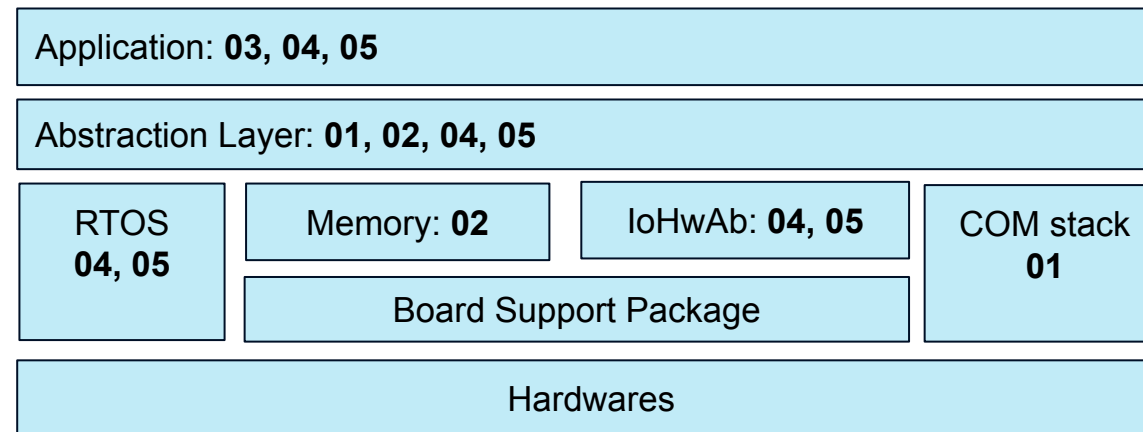
[REQ-02]: The system shall restore all the configuration parameters from the EEPROM at startup

[REQ-03]: There shall be a **watering** application to control soil moisture of the greenhouse

[REQ-04]: The system shall read <soil_moisture_level> from an analog soil moisture sensor every second

[REQ-05]: If <soil_moisture_level> goes above <soil_moisture_max>, the system shall open the <window>

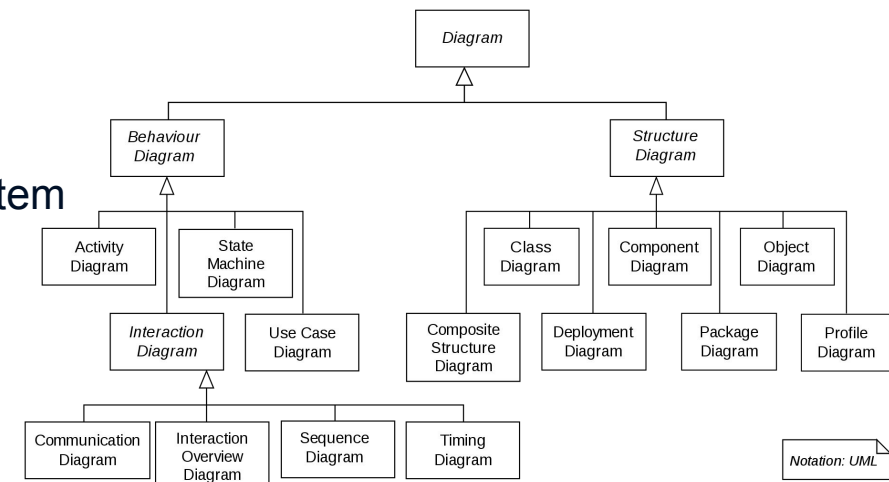
and turn the <fan> on until the soil moisture level falls below <soil_moisture_max> - 1 or for max. 30 min.



Unified Modeling Language

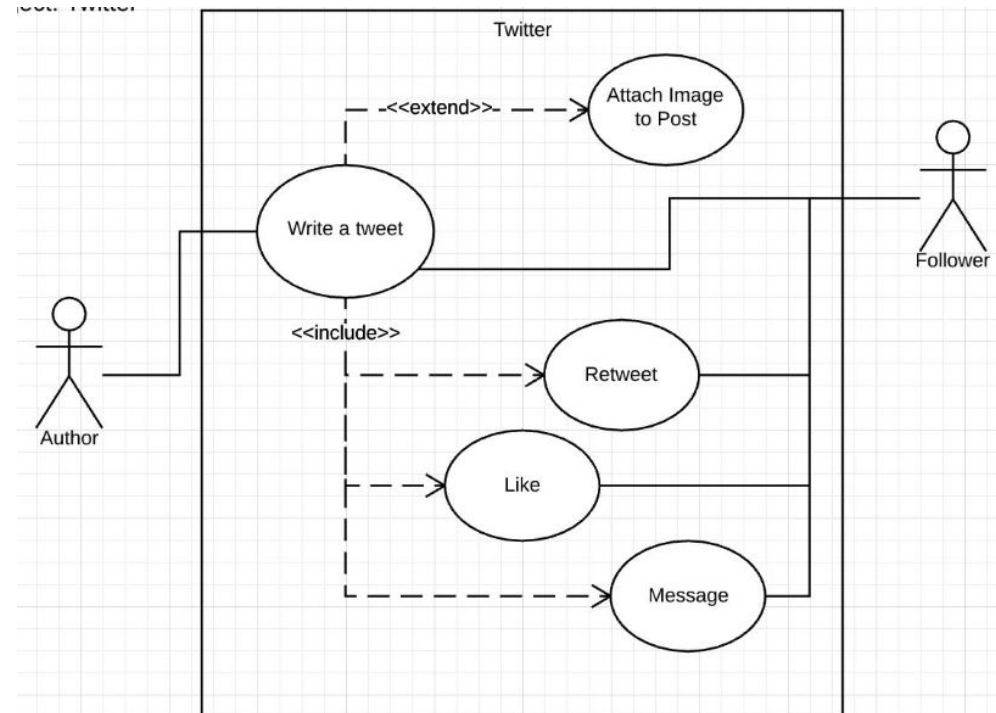
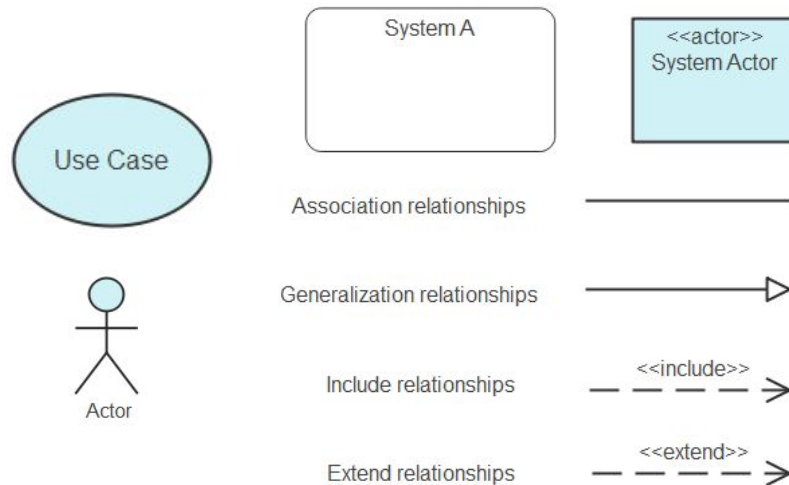
- ❖ **UML** is a standard modeling language consisting of an integrated set of diagrams
- ❖ Developed to help system and software developers for specifying, visualizing, constructing, and documenting the artifacts of softwares
- ❖ Developed by Rational Software in 1994 - 1995. Standardized by ISO in 2005.
- ❖ Has roots in Object Oriented Programming. Latest standard UML Specification 2.5.1
- ❖ Types of [UML Diagram](#):

- **Behavioral** diagrams: Used to describe and visualize the behavior, actors and building components of a system
 - E.g. Activity, use case, sequence diagrams
- **Structural** diagrams: Used to describe and visualize the structure of a system or process
 - E.g. Component and Class Diagram.



UML Use Case Diagram

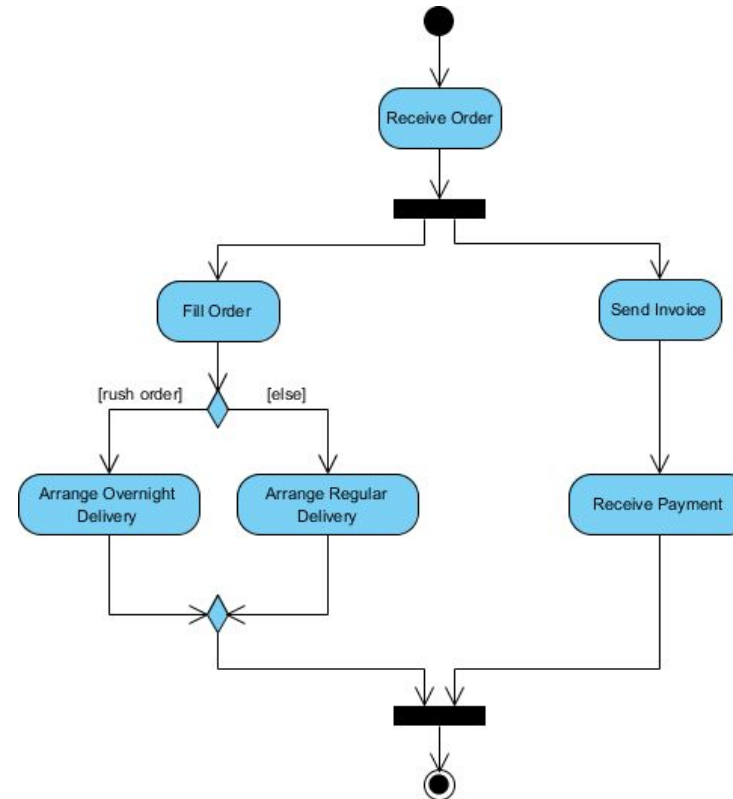
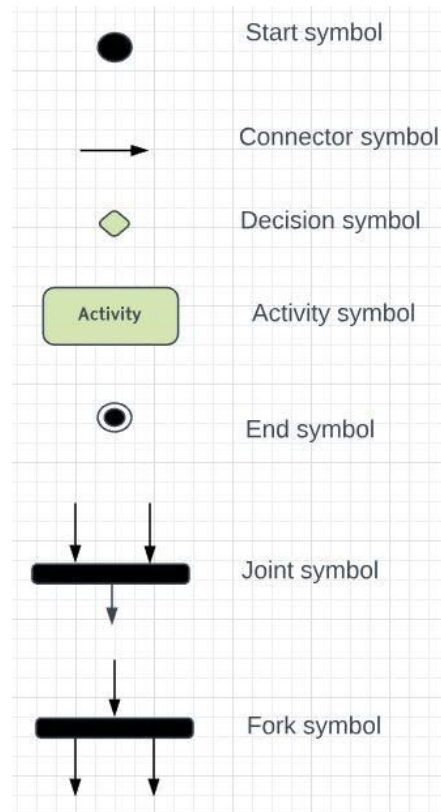
- ❖ **Use Case** diagrams are used to visualize user interactions with a system.
- ❖ Main symbols are **actors**, **system**, **use cases** and **relationships** [What is a use case diagram?](#)
 - An association relationship is a relationship between an actor and a use case.
 - A generalization relationship is a relationship in which a model element (the child) is based on another model element (the parent)



UML Activity Diagram

- ❖ Activity diagrams are used to visualize workflow with support of concurrency and iteration to describe what must happen in a system.

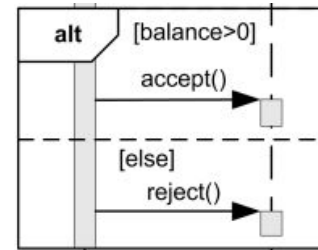
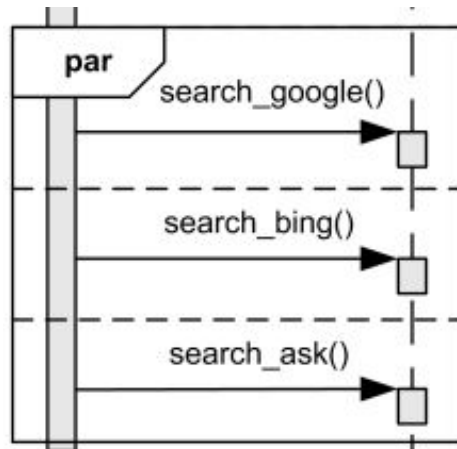
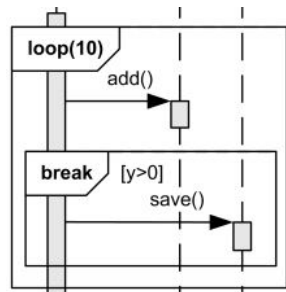
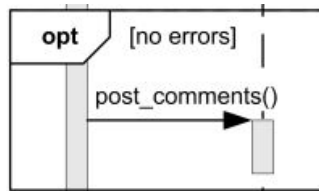
[What is an activity diagram?](#)



UML Sequence Diagram

- ❖ Sequence diagrams are used to visualize object interactions in a sequential order with support of combined fragments like loops, branches, concurrency.

- ❖ Some Sequence Diagram symbols

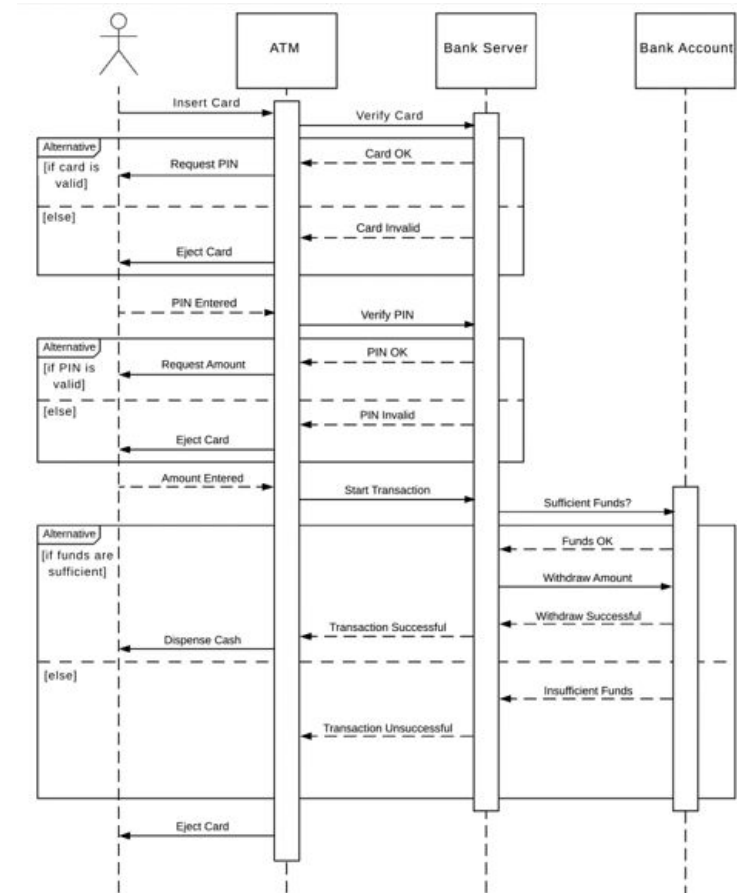


Message

Response

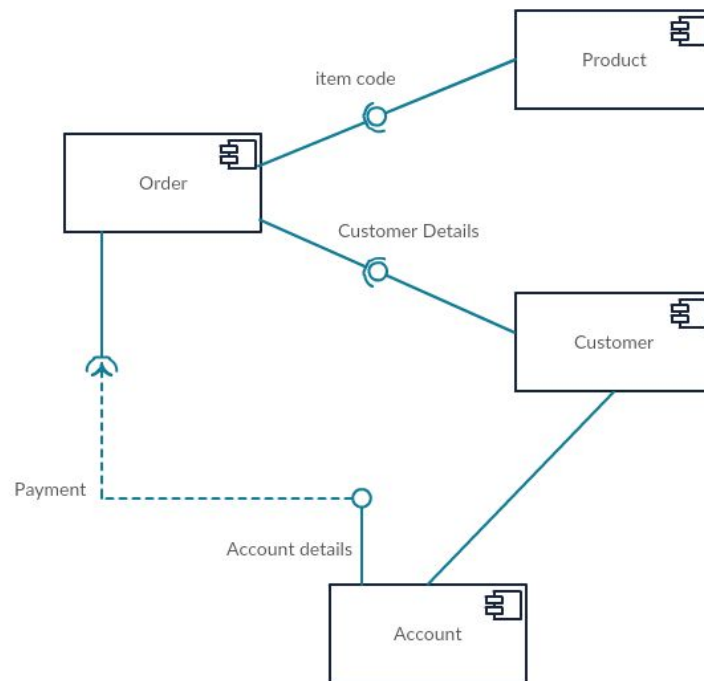
Lifeline

Activation

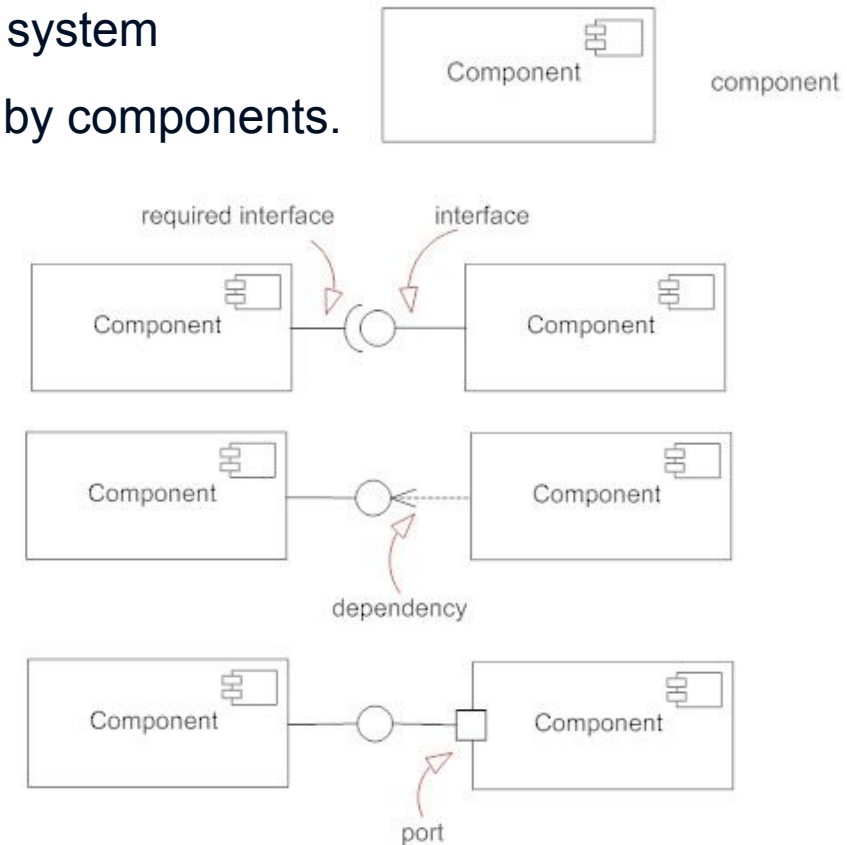
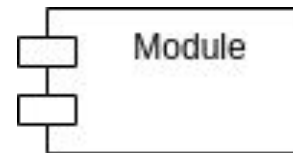


UML Component Diagram

- ❖ **Component** - Logical and physical unit block in a system
- ❖ **Interface** - A group of operations used or created by components.



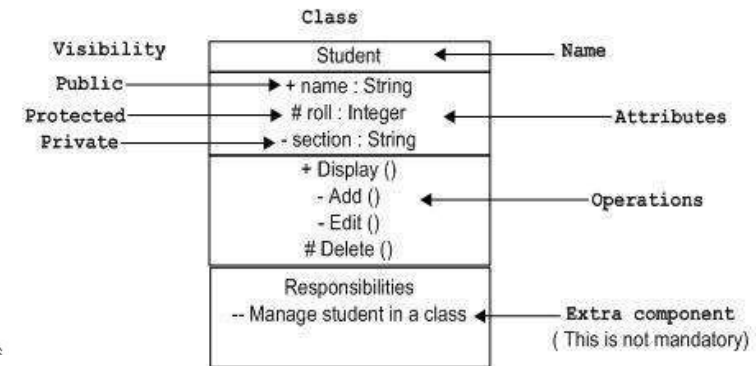
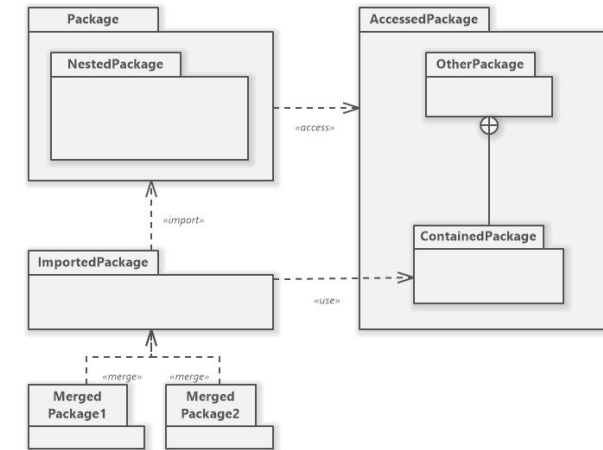
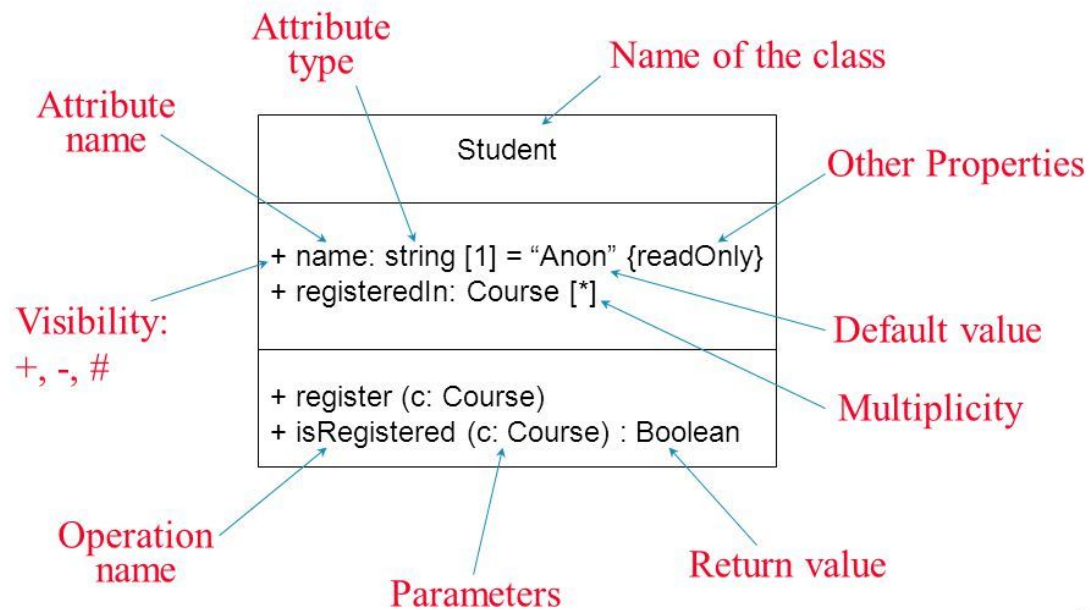
Component diagram of an online shopping system



A port is often used to expose required and provided interfaces

UML Class Diagram

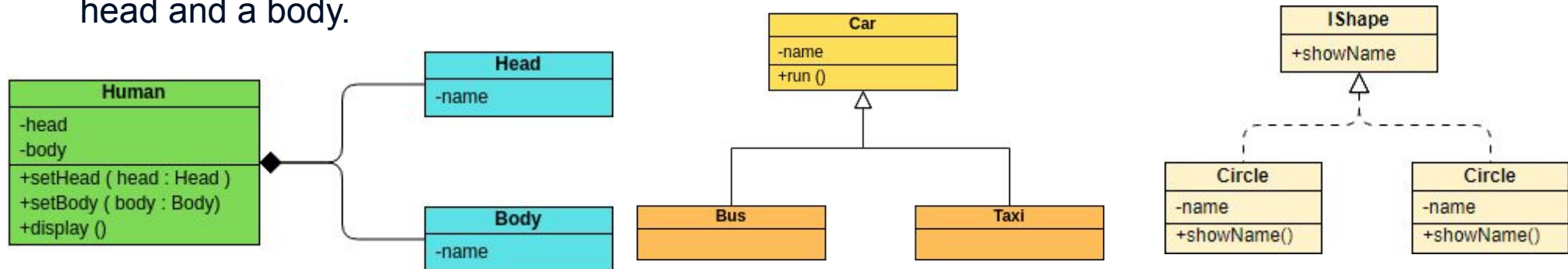
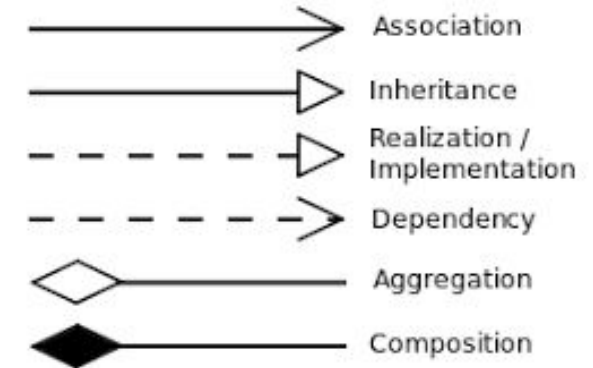
- ❖ A **Package** is a namespace used to group together elements that are semantically related and might change together.
- ❖ **Class diagrams** are used to model classes



UML Class Diagram

❖ Main relationships in UML class diagrams

- **Inheritance/generalization** is used for “is a/an” relationship
 - E.g. a car is a vehicle
- **Realization/Implementation** is used for relationship between interfaces or abstract classes and implementation classes.
- **Composition**: Is a "part-of" type of relationship. The relationship between the whole and the part, but the whole and the part cannot be separated. For example, a person is composed of a head and a body.

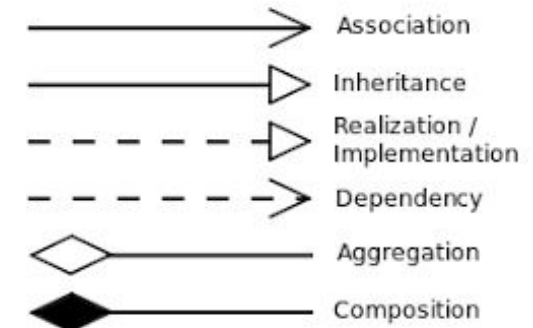


UML Class Diagram

❖ Main [relationships](#) in UML [class diagrams](#) ...

➤ **Association** is used for “is related to” relationship.

- It indicates that a property of a class holds a reference to an instance (or instances) of another class .
- For example: Cars and drivers; one car corresponds to a particular driver, and one driver can drive multiple cars.
- Four kinds of association: one-way, two-way, self-association, and multiple-number association.
- Bidirectional associations can have two arrows or no arrows.
- In a multiplicity relationship, you can add a number directly to the associated line to indicate the number of instance in the corresponding class.



UML Class Diagram

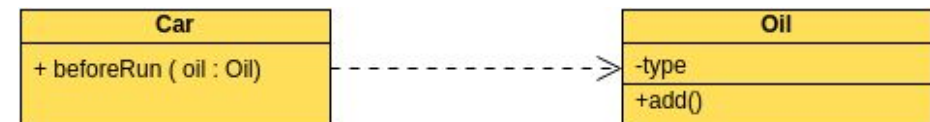
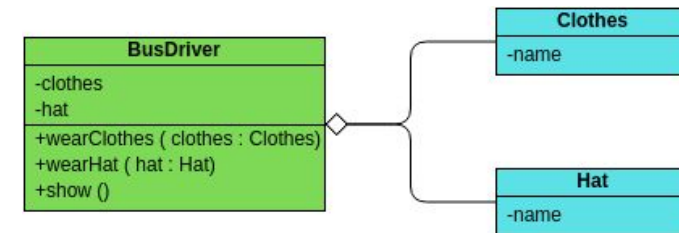
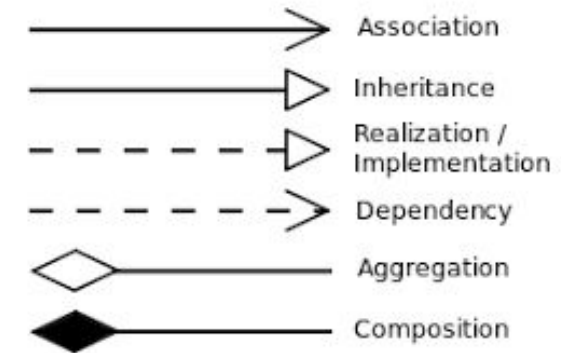
❖ Main relationships in UML class diagrams ...

➤ **Aggregation** is used for “has a/an” relationship.

- It is a one-way form of association
- The relationship between the whole and the part, and the whole and the part can be separated and the parts can exist independently from the overall object.
- E.g. A car needs wheels to function correctly, but a wheel doesn't always need a car.

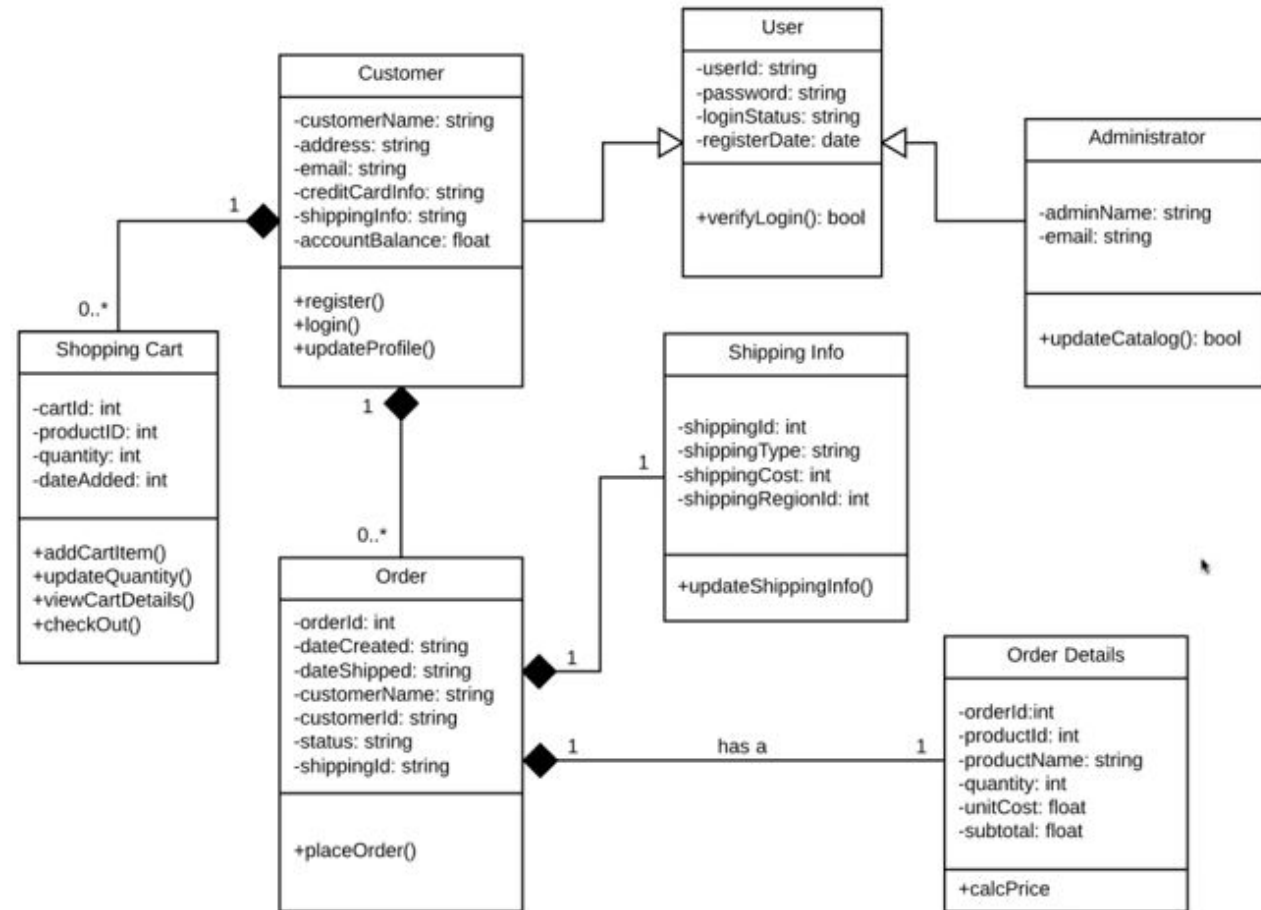
➤ **Dependency** is used for “uses” relationship.

- For example: Car relies on gasoline. If there is no gasoline, the car will not be able to drive.
- A change in class A causes a change in class B, then we say that class B depends on class A.



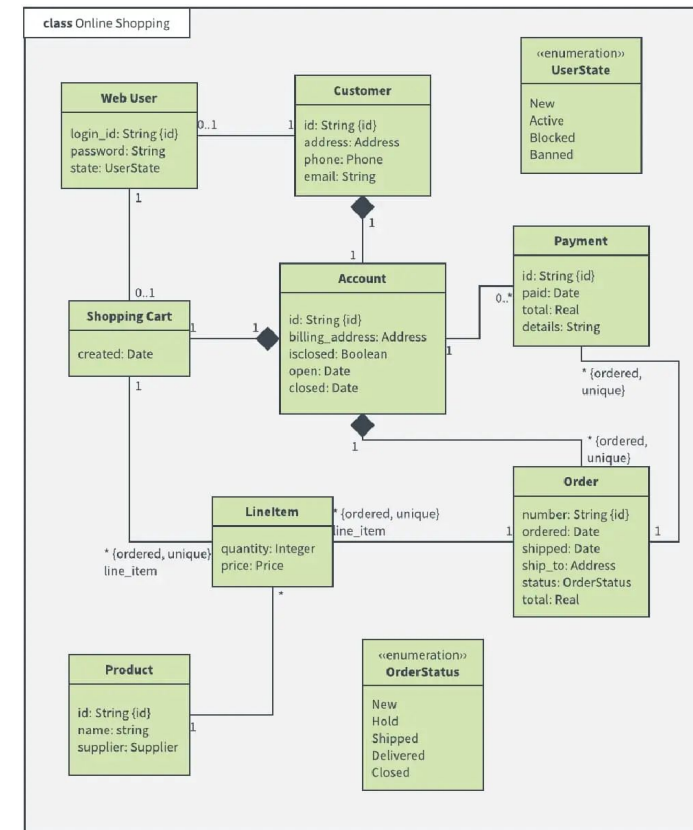
Unified Modeling Language

- ❖ There are many free desktop and online UML designer tools.
- ❖ Install the **Draw.io Integration** extension in vscode. File extension shall be .drawio



Software Design

- ❖ **Detailed(or Software) Design** is the specification of the internal elements of all the system components, their properties, relationships, processing, and often their algorithms and the data structures.
- ❖ **Domain or Conceptual Model:** to model properties and behaviors of classes and objects
 - Results of object-oriented analysis.
 - UML class diagram is used to draw Domain Models.
- ❖ **Use Case:** Scenarios for user and system interactions.
- ❖ **System Sequence Diagram:** Illustrates external actors, internal order, and messages passed for each use case.
- ❖ There are **Software Design Patterns:** general and reusable solutions to a commonly occurring problem, and can speed up software development



Software Design Patterns & Principles

❖ A design pattern is not ready to be translated to code, rather it is a template for how to tackle the same issue which happens often.

❖ Design Patterns should be adapted to solve problems, and they are classified in:

- **Behavioral** patterns deal with the communication among objects.
- **Structural** patterns deal with the relation between entities.
- **Creational** patterns deal with how the objects are created.
- **Concurrency** patterns deal with the multi-threaded programming.



Single Responsibility Principle

A class should have only a single responsibility (i.e. only one potential change in the software's specification should be able to affect the specification of the class)



Open / Closed Principle

A software module (it can be a class or method) should be open for extension but closed for modification.



Liskov Substitution Principle

Objects in a program should be replaceable with instances of their subtypes without altering the correctness of that program.



Interface Segregation Principle

Clients should not be forced to depend upon the interfaces that they do not use.



Dependency Inversion Principle

Program to an interface, not to an implementation.

❖ SOLID Design Principles (The 5 Principles of OOP Design)

- **Single Responsibility Principle**
- **Open Closed Principle**
- **Liskov Substitution Principle**
- **Interface Segregation Principle**
- **Dependency Inversion Principle**

Software Design Principles

- ❖ Principle of Abstraction recommends reduction of duplication wherever practical.
- ❖ Too much abstraction can increase complexity: **KISS!**
 - **Keep It Simple, Stupid** (KISS) is [a design principle](#). *Use inheritance only if the relationship is "is a/an".*
 - It means that designs should be as simple as possible.
- ❖ Abstraction which is only used once is not good: **YAGNI!**
 - **You Ain't Gonna Need It** (YAGNI) is an XP principle advocating implementation if only necessary.
- ❖ **Rule of three** (a code refactoring rule) is an abstraction principle in code.
 - It suggests to refactor a code which is used(copied) more than twice into a function.
- ❖ **DRY** is a generalization of abstraction principle.
 - **Don't Repeat Yourself** (DRY) states "Every piece of knowledge must have a single and unambiguous representation within a system".
- ❖ **General Responsibility Assignment Software Patterns** ([GRASP](#))
 - Guidelines for assigning responsibilities to classes and objects

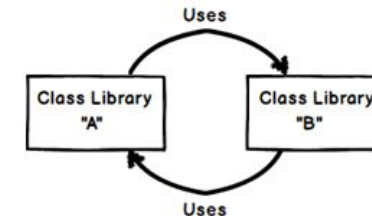
Software Design - Anti-patterns

- ❖ Anti-pattern is a term that describes how NOT to solve recurring problems in your code.
 - Anti-patterns are considered as bad software design, and they are usually ineffective
- ❖ **Cart before the horse:** prioritizing tasks irrationally
- ❖ **Analysis Paralysis:** A scenario in which overthinking “paralyzes” decision making.
 - Iterative, incremental, and agile have come to rescue!
 - Try to understand the main objectives.
 - Start early, prioritize tasks appropriately and set a deadline!
 - Limit the inputs, intentionally.
 - Decompose problem and iterate on solutions!
 - Do not worry about being wrong.
 - Ask help.
- ❖ **Overengineering:** making a project more robust and complex than what it is needed



Software Design - Anti-patterns

- ❖ **Action at a distance:** unexpected interaction between widely separated parts of a system
- ❖ **Scope/Feature/Requirement creep:** adding new features to the project after the original requirements have been drafted and accepted
- ❖ **Call super:** a subclass overrides a method from its superclass but simply calls the superclass's implementation without adding meaningful behavior
- ❖ **Circular dependency:** introducing unnecessary direct or indirect mutual dependencies between objects or software modules
- ❖ **God object:** too many functions in a single part of the design (class)
- ❖ **Object orgy:** failing to properly encapsulate objects permitting unrestricted access to their internals
- ❖ **Input kludge:** a poor user interface that either rejects or incorrectly handles valid input.

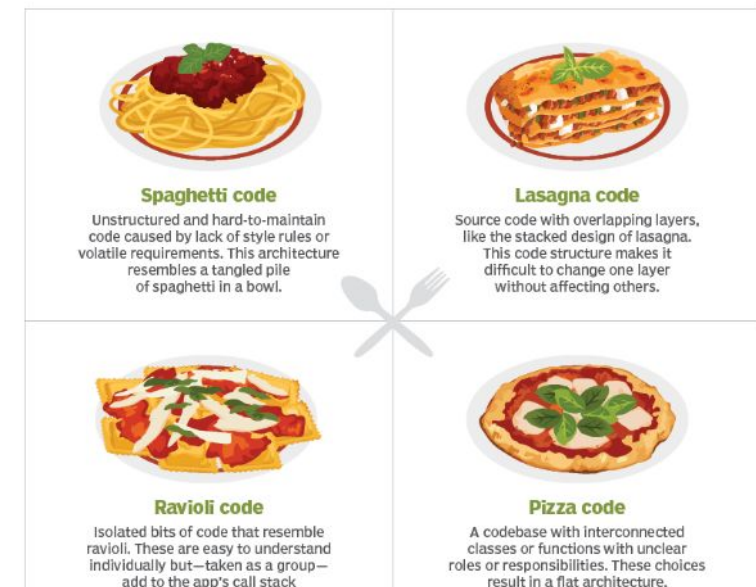


Software Design - Anti-patterns

- ❖ **Race Hazard:** It occurs when there is an assumption that two events cannot happen at once, but they do, and they break the solution. [🔗 Anti-patterns and How Not To Build Software](#)
- ❖ **Premature Optimization:** too early optimization for efficiency before ensuring that it is required and sacrificing maintainability and good design
- ❖ **Magic numbers & Magic strings**
- ❖ **Hard code:** Embedding assumptions about the environment of a system in its implementation
- ❖ **Ravioli:** well-structured classes that are easy to understand in isolation, but difficult to understand as a whole.
- ❖ **Spaghetti:** unstructured and difficult-to-maintain
- ❖ **Lasagna:** too many layers of inheritance
- ❖ **Pizza:** Unclear roles which results in a flat architecture.



The pasta theory of programming



Software Design

❖ Some useful links

- [Software Design Process – Software Engineering](#)
- [Why Software Design Is Important](#)
- [Introduction of Software Design Process](#)
- [Software Design Basics](#)
- [AUTOSAR Classic Platform](#)
- [AUTOSAR Adaptive Platform](#)
- [AUTOSAR \(Automotive Open System Architecture\)](#)
- [AUTOSAR Explained Layered Software Architecture](#)
- [AUTOSAR Basic Software and Beyond](#)
- [Introduction to AUTOSAR](#)
- [C++ Design Patterns](#)